LAPORAN PRAKTIKUM

Proyek 3 – Cookies, Session, dan Local Storage

WEEK 5



Repository Github : https://github.com/Rafreaakk/Proyek-3-Week-5.git

Disusun Oleh:
Muhammad Rafi Ardiyansyah – 251511051 – 2B_D3
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

LAPORAN PRAKTIKUM MODUL 4

1. Tugas-1-Login

A. Deskripsi Implementasi

Praktikum ini mengimplementasikan sistem autentikasi pengguna secara manual menggunakan Laravel 13. Sistem terhubung dengan *database* MySQL (`db_tugas`) dan menggunakan tabel `users` yang telah dimodifikasi untuk menggunakan kredensial `username` alih-alih `email`.

Keamanan sistem dibangun melalui dua pendekatan utama:

1. **Proteksi Data:** Kata sandi (*password*) pengguna tidak disimpan dalam bentuk teks biasa, melainkan dienkripsi secara satu arah menggunakan algoritma *hashing* sebelum disimpan ke *database*.
2. **Proteksi Rute:** Akses antarmuka dikendalikan sepenuhnya melalui *middleware*. Halaman `/dashboard` dilindungi oleh *middleware* `auth` sehingga hanya dapat diakses oleh pengguna terautentikasi, sedangkan form `/login` dilindungi oleh *middleware* `guest` untuk mencegah pengguna yang sudah masuk mengakses halaman login kembali.

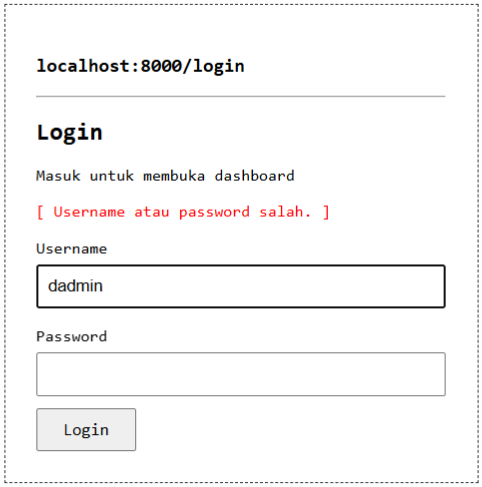
B. Daftar Screenshot

- **Isi table users (Password berupa hash) :**

	id	nama_lengkap	username	password	created_at	updated_at
Edit Copy Delete	1	ableh	admin	$2y$12$fPJboNuK910xET3iO/GXU.XcaajCR9Tnn9yOBDYiHBC...	2026-10-06 10:42:25	2026-10-06 10:42:25
Edit Copy Delete	2	hilmi kautsar	hilmi	$2y$12$qUXK.KKMNSmH5GiSQsMkSOJ2OIOpJixF6/DUWTcKj5Y...	2026-10-06 10:42:25	2026-10-06 10:42:25

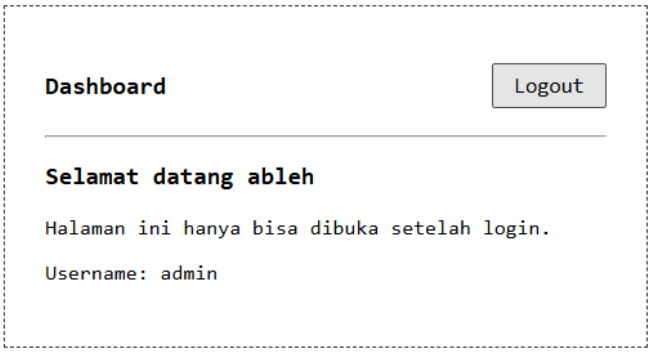
Data pengguna berhasil dimasukkan ke dalam database menggunakan Seeder. Kolom password telah dienkripsi menggunakan algoritma *hashing* bawaan Laravel (Bcrypt) demi keamanan.

- **Login Gagal (Menampilkan form dengan pesan peringatan) :**



Halaman antarmuka login yang dibuat menggunakan Blade template, tempat pengguna memasukkan *username* dan *password*. Rute ini dilindungi oleh middleware guest.

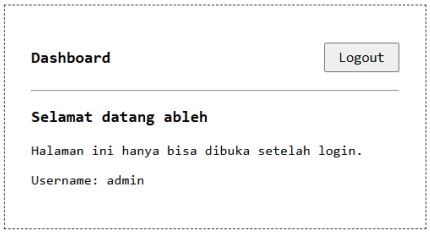
- **Login Berhasil (Proses form diisi dengan benar) :**



Pengujian keamanan ketika pengguna memasukkan kredensial yang tidak valid. Sistem menolak akses dan mengembalikan pesan *error*.

- **Dashboard (Halaman /dashboard)**





Setelah kredensial diverifikasi, pengguna diarahkan ke halaman Dashboard. Halaman ini berhasil menarik data pengguna yang sedang *login* (seperti Nama Lengkap dan Username) melalui *facade* Auth::user().

- **/dashboard dibuka di incognito, maka otomatis akan kembali ke halaman /login**





Bukti bahwa middleware auth berfungsi dengan baik. Pengguna yang belum memiliki sesi *login* tidak akan bisa mengakses rute yang diproteksi.

- **Setelah Logout**





C. Jawaban Pertanyaan

a) **Mengapa password harus di-hash? Lihat kolom password di tabel users dan jelaskan isinya!**

Password wajib di-hash agar tidak tersimpan dalam bentuk teks biasa di dalam database. Hal ini krusial untuk melindungi kerahasiaan kata sandi pengguna jika sewaktu-waktu terjadi peretasan atau kebocoran data. Pada kolom password di tabel users, isinya bukan lagi password asli yang diketik pengguna, melainkan deretan karakter acak (ciphertext) hasil enkripsi algoritma hashing bawaan Laravel yang tidak bisa di-dekripsi (dikembalikan) ke bentuk aslinya.

b) **Mengapa logout memakai metode POST dengan @csrf, bukan link biasa?**

Logout menggunakan metode POST dan perlindungan @csrf (Cross-Site Request Forgery) untuk mencegah serangan di mana situs web jahat atau skrip pihak ketiga memaksa pengguna untuk logout secara sewenang-wenang tanpa sepengetahuan mereka. Jika menggunakan link biasa (GET), aksi logout akan sangat mudah dipicu hanya dengan mengakses tautan tersebut.

c) **Apa fungsi middleware auth dan guest? Apa yang terjadi jika /dashboard dibuka tanpa login?**

- **Fungsi auth**: Memastikan rute tersebut hanya bisa diakses oleh pengguna yang sudah memiliki sesi login yang sah.

- **Fungsi guest**: Memastikan rute tersebut (seperti halaman form login) hanya bisa diakses oleh pengunjung tamu yang *belum* login. Jika halaman /dashboard dibuka secara paksa tanpa login, middleware auth akan mencegat permintaan tersebut dan secara otomatis mengalihkan pengguna kembali ke halaman /login.

d) **Apa fungsi session()->regenerate() setelah login berhasil?**

Fungsi ini digunakan untuk mencegah serangan *Session Fixation*. Sesaat setelah kredensial pengguna divalidasi dan login berhasil, Laravel akan memperbarui dan membuat ulang ID sesi (*session ID*) dengan yang baru. Dengan demikian, ID sesi lama yang mungkin sudah disusupkan atau diketahui oleh peretas menjadi tidak berlaku, sehingga mereka tidak bisa membajak sesi login pengguna tersebut.

2. Tugas2 – Keranjang – Belanja

A. Deskripsi Implementasi

Praktikum ini mengimplementasikan aplikasi keranjang belanja sederhana bertema "Toko Celana Denim" menggunakan kerangka kerja Laravel 13. Data produk (ID, nama, harga, dan stok) dikelola secara dinamis melalui data MySQL pada tabel barang.

Mekanisme penyimpanan keranjang belanja yang dipilih pada implementasi ini adalah **Session**. Data yang disimpan di dalam *session* hanyalah ID Produk dan Jumlah (Qty). Pengambilan atribut nama dan harga barang selalu diproses secara *real-time* dari *database* setiap kali halaman keranjang dimuat untuk mencegah manipulasi sisi klien.

B. Daftar Screenshot

- **Data Produk :**

	id	nama	harga	stok	created_at	updated_at
Edit Copy Delete	1	Raw Denim 22Oz Zero Zeke	1600000	10	2026-10-08 10:43:19	2026-10-08 10:43:19
Edit Copy Delete	2	Raw Denim 17Oz IndiBrown	1200000	15	2026-10-08 10:43:19	2026-10-08 10:43:19
Edit Copy Delete	3	Raw Denim 14Oz Indigo	800000	20	2026-10-08 10:43:19	2026-10-08 10:43:19
Edit Copy Delete	4	Japanese Selvedge Denim 16oz	1000000	12	2026-10-08 10:43:19	2026-10-08 10:43:19
Edit Copy Delete	5	Loose Denim 120z IndiBrown	900000	18	2026-10-08 10:43:19	2026-10-08 10:43:19

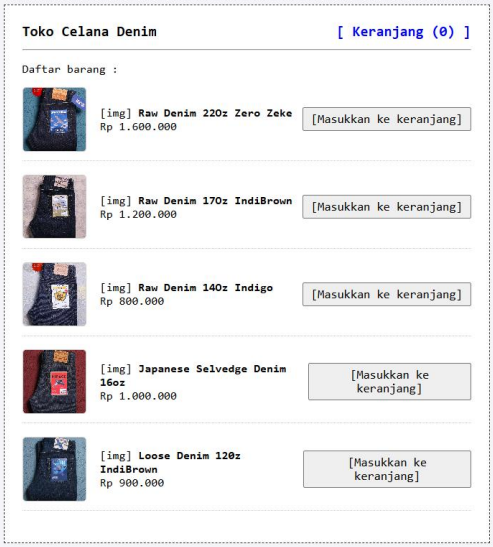
Data pengguna berhasil dimasukkan ke dalam database menggunakan Seeder. Kolom password telah dienkripsi menggunakan algoritma *hashing* bawaan Laravel (Bcrypt) demi keamanan.

- **Isi table users (Password berupa hash) :**

	id	nama_lengkap	username	password	created_at	updated_at
Edit Copy Delete	1	ableh	admin	$2y$12$fPJboNuK910xET3iO/GXU.XcaajCR9Tnn9yOBDYiHBC...	2026-10-06 10:42:25	2026-10-06 10:42:25
Edit Copy Delete	2	hilmi kautsar	hilmi	$2y$12$qUXK.KKMNSmH5GiSQsMkSOJ2OIOpJixF6/DUWTcKj5Y...	2026-10-06 10:42:25	2026-10-06 10:42:25

Data pengguna berhasil dimasukkan ke dalam database menggunakan Seeder. Kolom password telah dienkripsi menggunakan algoritma *hashing* bawaan Laravel (Bcrypt) demi keamanan.

- **Halaman daftar barang :**



Halaman antarmuka yang menampilkan katalog Toko Celana Denim dengan tombol ”Masukkan ke keranjang”.

- **Halaman Keranjang :**



Menampilkan halaman keranjang saat belum ada barang yang dimasukkan



Menampilkan halaman keranjang saat sudah ditambahkan barang.

- **Fitur Wajib**



Menambahkan produk yang sama dan hasil jumlahnya bertambah



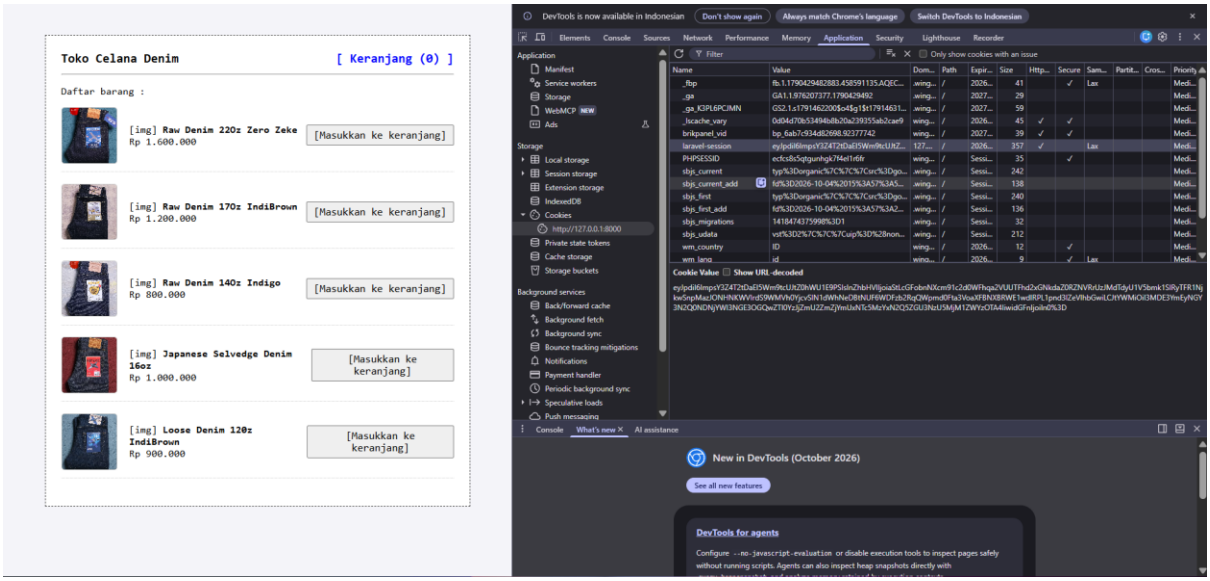
Tampilan ketika keranjang dikosongkan

- **Ketahanan data**



Setelah halaman page di refresh, barang yang sudah di keranjang tidak hilang begitu saja.

- **Lokasi Data di DevTools :**



Menampilkan tab Application > Cookies di DevTools browser untuk memperlihatkan keberadaan cookie sesi (Laravel _session).

D. Jawaban Pertanyaan

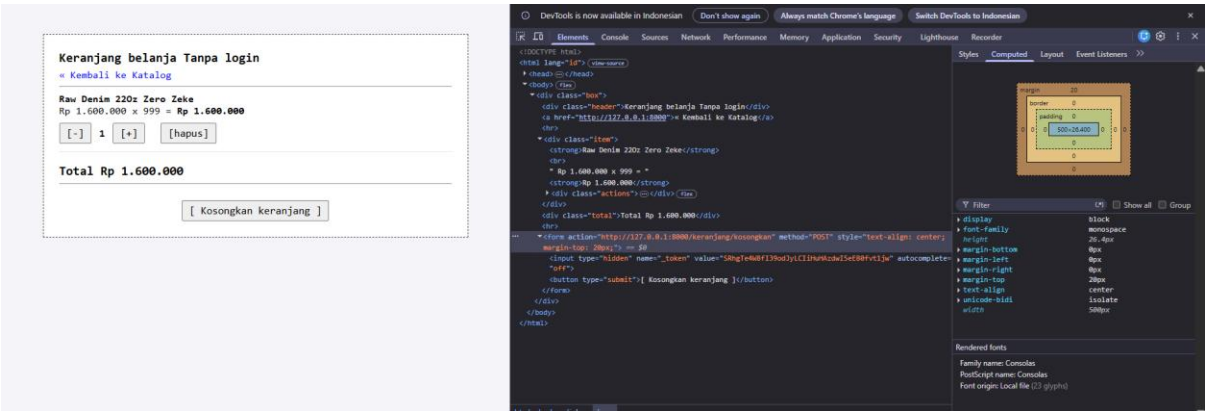
a) **Mekanisme apa yang Anda pilih (session, cookies, atau Local Storage), dan mengapa?**

Mekanisme yang dipilih adalah **Session**. Alasannya, keranjang belanja memuat data yang berkaitan dengan transaksi, sehingga penyimpanannya harus aman. Menyimpan data di sisi *server* (Session) jauh lebih aman daripada di sisi klien karena pengguna tidak bisa melihat secara langsung bentuk struktur keranjangnya dan memanipulasi isinya (seperti jumlah atau harga barang) melalui *DevTools* browser.

b) **Sebutkan satu kelebihan dan satu kekurangan pilihan Anda dibanding dua mekanisme lainnya.**

- Kelebihan: Tingkat keamanan tinggi. Klien hanya memegang kunci referensi (cookie ID session), sedangkan data asli keranjang tersimpat di memori server, ini mencegah manipulasi client dibandingkan Cookies biasa atu *Local Storage.*
- Kekurangan: Membutuhkan lebih banyak sumber daya server, Karena setiap keranjang aktif disimpan di memori server atau basis data, beban server akan meningkat secara signifikan jika ada ribuan pengunjung daring secara bersamaan.

c) **Ubah data keranjang secara manual lewat DevTools (misalnya jumlah menjadi 999) bila memungkinkan pada mekanisme Anda. Apa yang terjadi? Jika tidak bisa diubah, jelaskan alasannya.**



Data keranjang seperti jumlah barang tidak bisa diubah secara manual lewat DevTools. Alesannyak arena mekanisme session menyimpan data secara fisik di dalam server.

Dalam pengujian, Jumlah Denim diubah menjadi 999 sehingga aplikasi tidak membaca input perubahan tersebut dan totalnya tetap seperti yang ada pada session yakni 1. Namun bisa saja Total tersebut diubah oleh client dan melakukan checkout.

d) **Mengapa harga tidak boleh dipercaya dari data yang dikirim atau disimpan di browser? Apa yang harus dilakukan server saat checkout?**

Data yang dikirim dari browser sangat rentan dimodifikasi. Pengguna nakal dapat menggunakan fitur *Inspect Element*, mencegat permintaan HTTP jaringan , atau mengubah *Local Storage* untuk memalsukan nilai harga (misalnya mengubah celana Rp1.600.000 menjadi Rp1). Oleh karena itu, saat proses *checkout*, *server* **hanya boleh menerima data ID Produk dan Qty**. Untuk perhitungannya, *server* wajib melakukan query ulang ke dalam *database* untuk mendapatkan harga barang yang asli dan sah.

e) **Apa yang terjadi pada keranjang jika pengguna membuka situs dari browser lain? Jelaskan sesuai pilihan Anda.**

Jawabannya Isi keranjang belanja **akan kosong**. Mekanisme *Session* bergantung pada *cookie* ID unik yang dititipkan pada satu sesi browser tertentu. Jika pengguna/klien membuka situs melalui browser yang berbeda (misalnya dari Chrome pindah ke Firefox), browser baru tersebut tidak memiliki *cookie* ID yang lama. Akibatnya, *server* akan mengidentifikasinya sebagai pengunjung baru dan membuatkan sesi keranjang yang baru dan kosong.

3. Tugas3 – Toko_Online

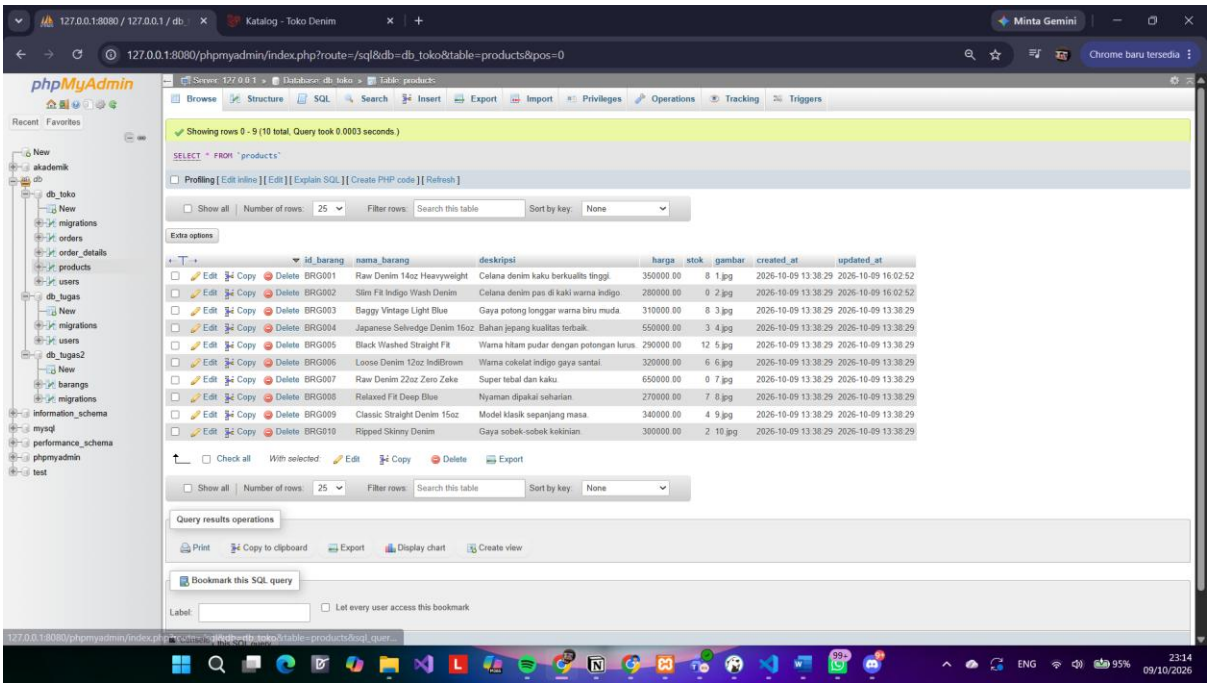
A. Deskripsi Implementasi

Pada praktikum kali ini menmbuat aplikasi e-commerce sederhana dengan aturan bisnis yang ketat. Katalog barang menampilkan minimal 10 jenis produk (termasuk dengan produk dengan stok kosong). Klien bebas melihat daftar barang, namun diwajibkan untuk login sebelum dapat memasukkan barang ke keranjang atau melakukan checkout.

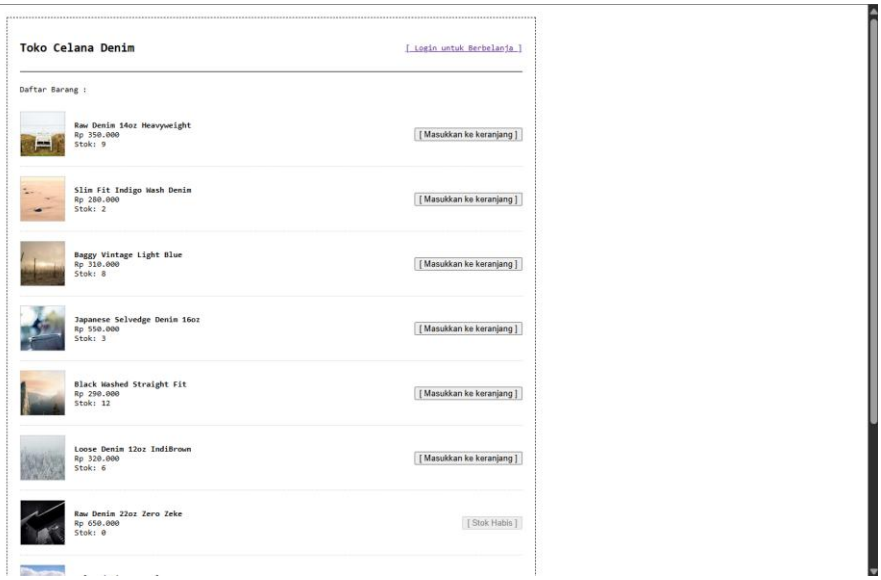
Saat klien melakukan checkout, sistem secara otomatis memproses transaksi menggunakan mekanisme Database Transaction, menyimpan rincian pesanan, mengurangi stok barang yang dibeli dari database, serta mengosongkan keranjang belanja klien.

B. Daftar Screenshot

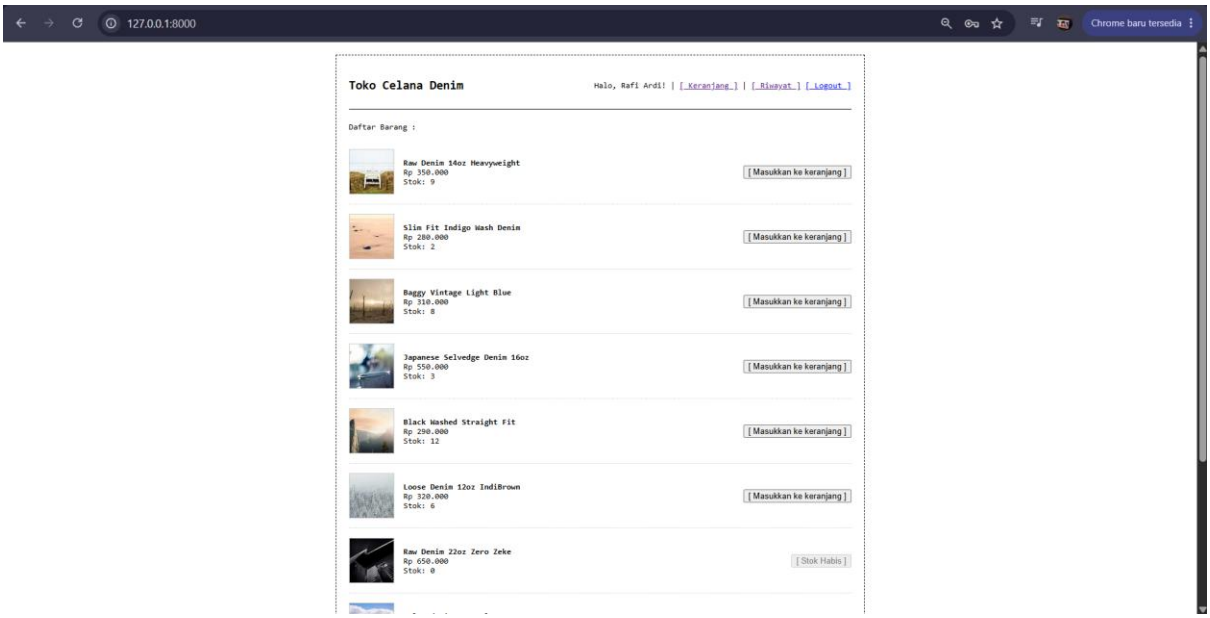
- **Daftar Produk di Database :**



- **Katalog Barang (Guest & Auth) :**
  Sebelum Login (Guest) :

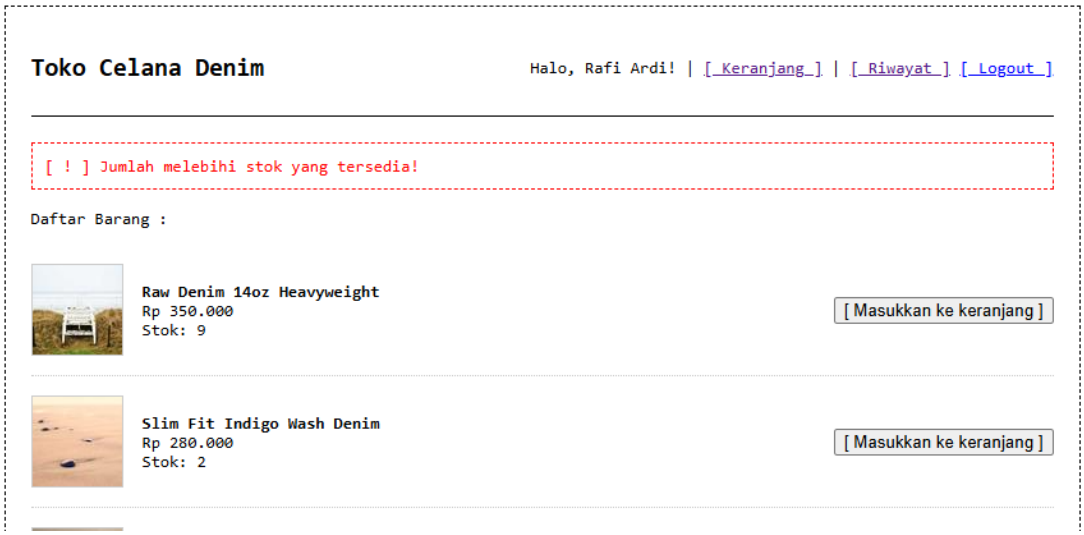


Setelah Login (Guest) :



Menunjukkan perbedaan akses menu navigasi antara pengunjung guest dan pengguna terautentikasi, serta memperlihatkan tombol aksi yang otomatis dinonaktifkan pada produk dengan stok habis.

- **Validasi Keranjang :**



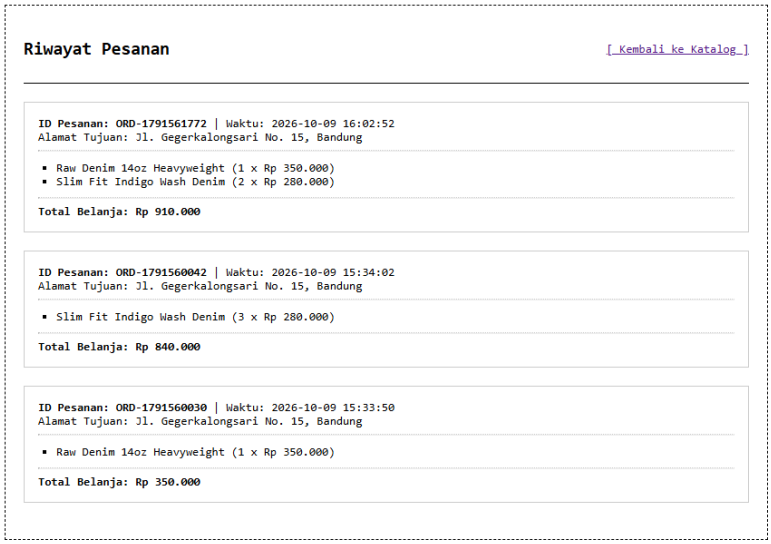
Memperlihatkan respons sistem berupa pesan peringatan ketika pengguna mencoba menambahkan barang ke keranjang melebihi batas stok yang tersedia di pangkalan data.

- **Keranjang Belanja :**



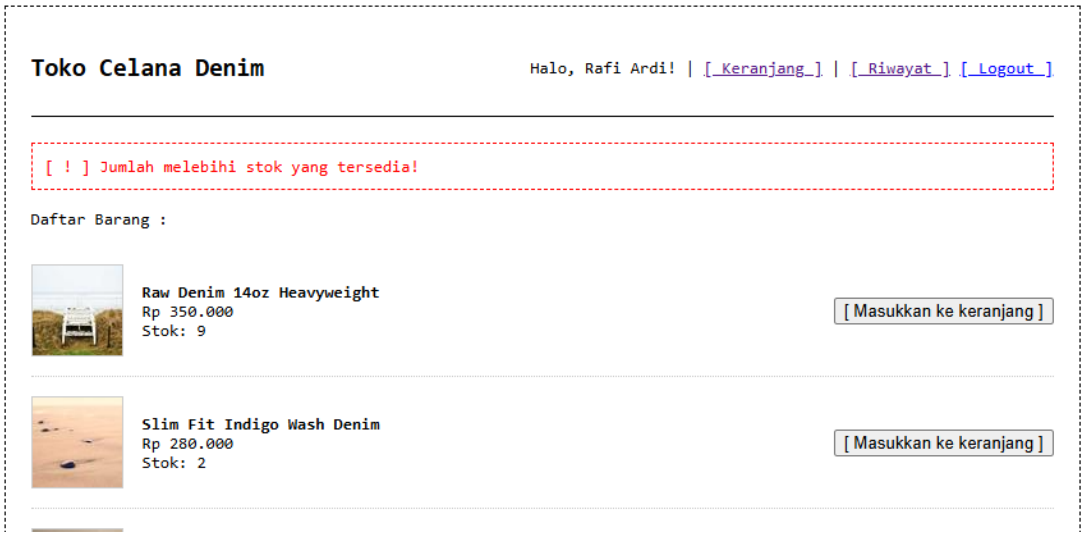
Menampilkan antarmuka keranjang belanja yang berisi rincian item pilihan pengguna, lengkap dengan fitur ubah kuantitas, kalkulasi subtotal, dan total harga keseluruhan.

- **Riwayat Pesanan :**

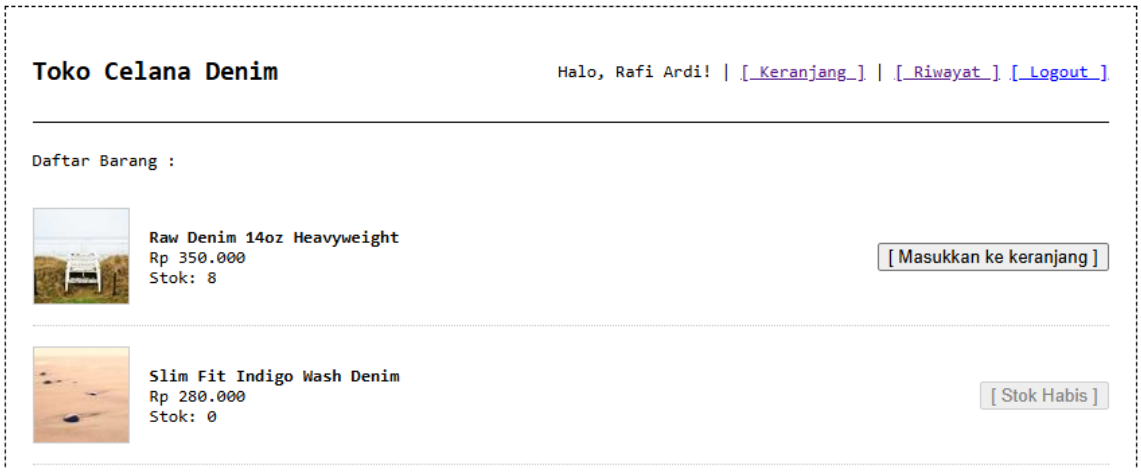


Menampilkan bukti transaksi yang berhasil diproses, memuat informasi detail seperti ID pesanan unik, alamat pengiriman, rincian barang yang dibeli, dan total pembayaran.

- **Bukti Pengurangan stock :**
  Sebelum Checkout :



Setelah Checkout :



Menunjukkan perbandingan data stok barang yang membuktikan bahwa sistem telah berhasil mengurangi stok secara permanen setelah pengguna melakukan *checkout*.

C. Alur Sistem dan Database

Pengguna melakukan login
↓
Melihat daftar barang di Katalog
↓
Memilih produk dan memasukkannya ke keranjang
↓
Sistem memvalidasi stok barang di database (Tabel products)
↓
Pengguna mengatur jumlah barang di halaman keranjang
↓
Menekan tombol "Lanjutkan Checkout"
↓
Sistem memulai Database Transaction
↓
Membuat data pesanan dan menyimpan alamat otomatis (Tabel orders)
↓
Menyimpan rincian harga dan jumlah barang (Tabel order_details)
↓
Sistem mengurangi jumlah stok barang secara permanen (Tabel products)
↓
Sistem menghapus data sementara keranjang belanja (Session)

↓

Mengarahkan pengguna ke halaman Riwayat Pesanan

↓

Pesanan berhasil tampil pada Riwayat

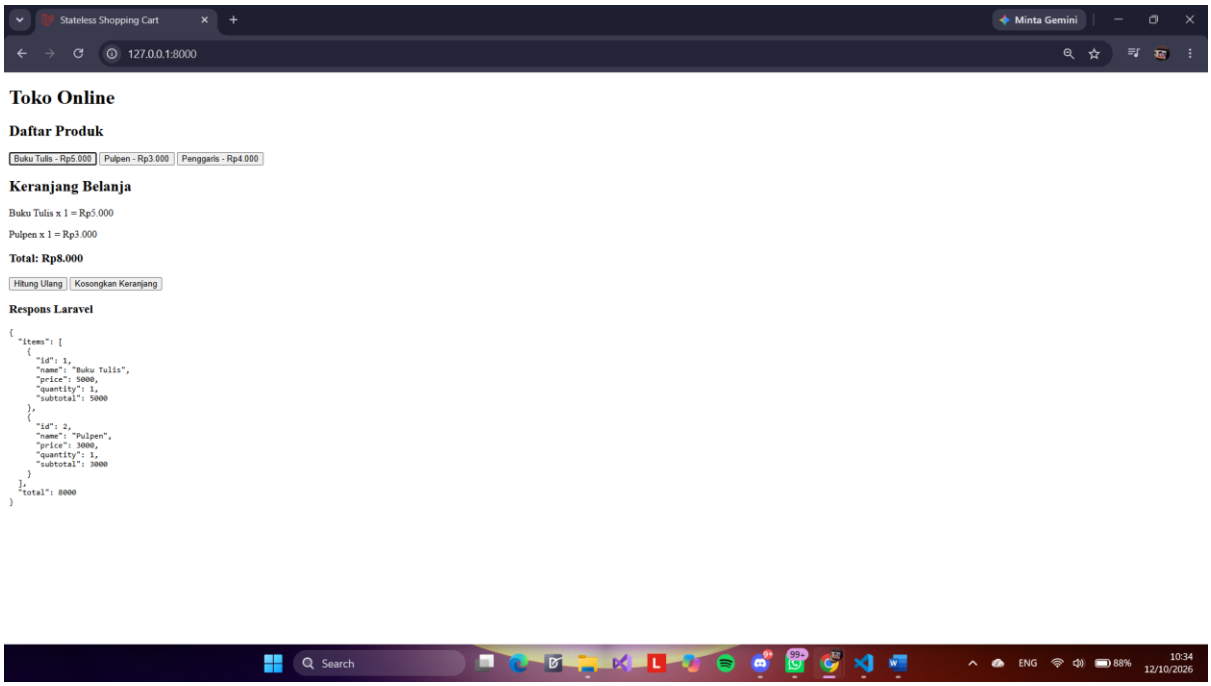
REFLEKSI HASIL PENGERJAAN

Dari praktikum ini saya mendapatkan pengalaman dalam membangun aplikasi web menggunakan Laravel. Mulai dari penguasaan keamanan autentikasi dan middleware (Tugas 1), pengelolaan keranjang belanja menggunakan Session yang aman dari manipulasi klien (Tugas 2), hingga integrasi keduanya menjadi sistem toko online utuh (Tugas 3). Penerapan logika bisnis tingkat lanjut, seperti relasi basis data menggunakan Primary Key string dan pengamanan proses checkout dengan Database Transaction, memastikan integritas data pesanan dan stok selalu sinkron.

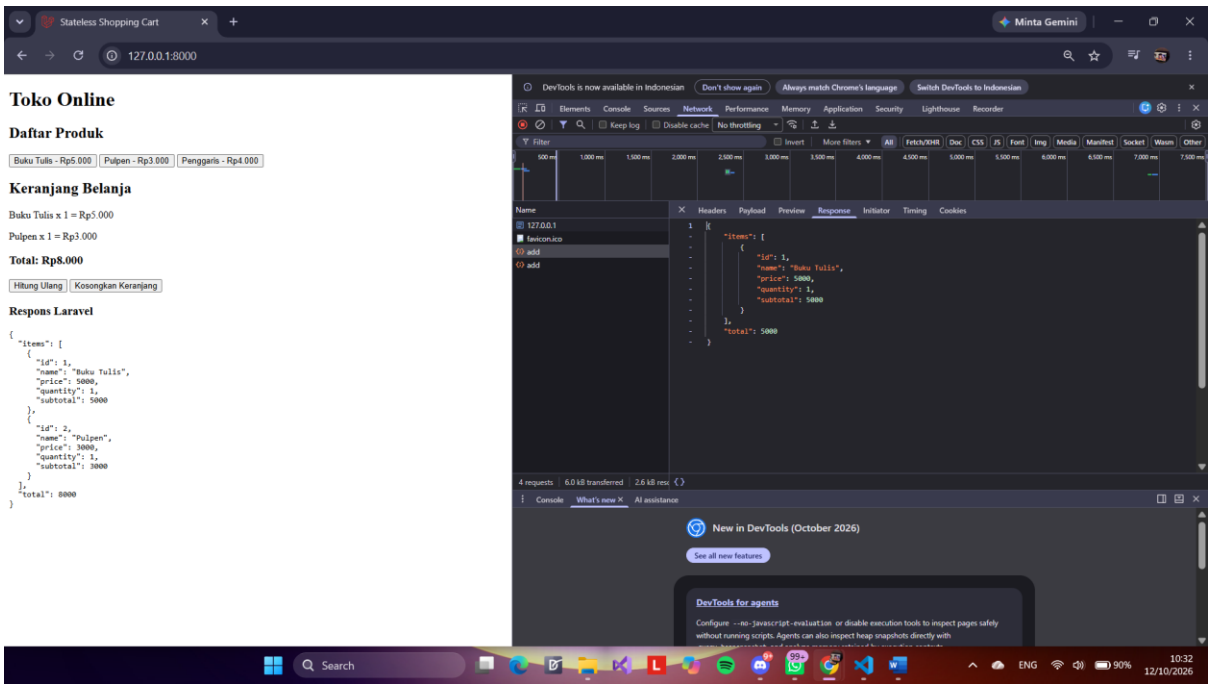
Modul 4 Bagian 2 — Stateless Shopping Cart.

Hasil Eksperimen :

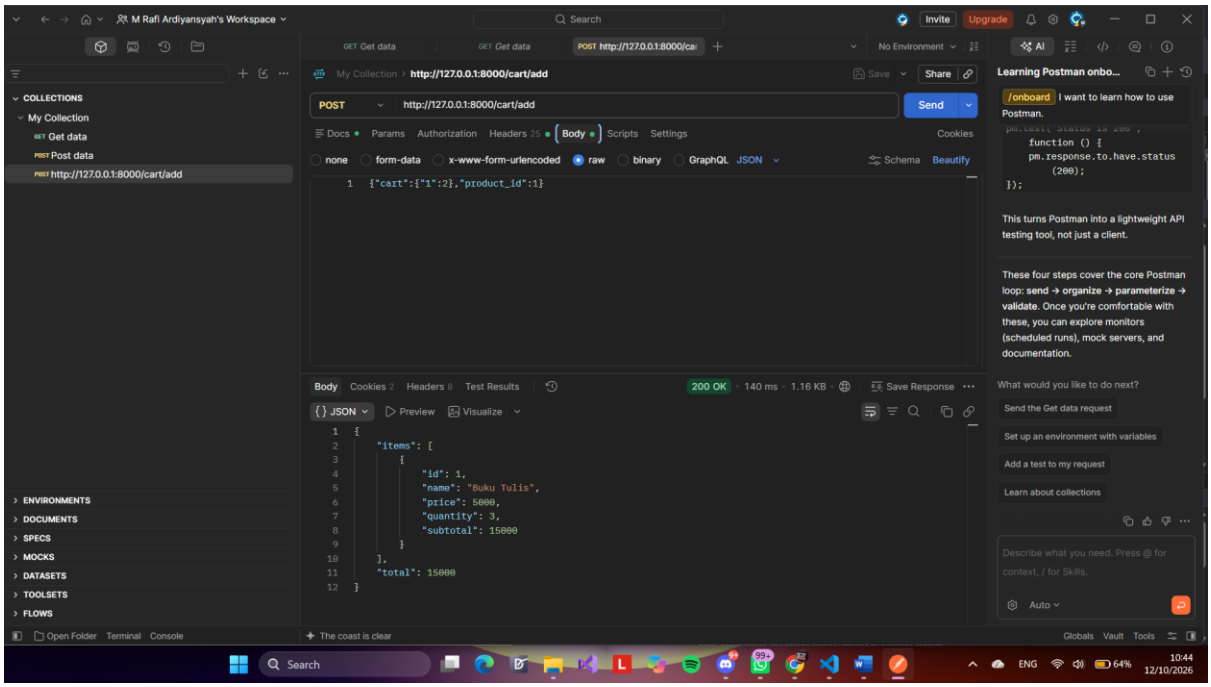
- Tangkapan Layar utama



- Request Payload/Response



- Payload dari Postman



Diskusi Dan Evaluasi

1. Di baris kode mana browser mengirim state keranjang ke Laravel?
   body: JSON.stringify(payload) di dalam fungsi post() (menggunakan fetch API). Pemanggilan ini terjadi saat fungsi add() menjalankan post('/cart/add', ...) dan fungsi recalculate() menjalankan post('/cart/update', ...).
2. Mengapa controller tidak perlu membaca session untuk menghitung keranjang?
   Karena aplikasi menerapkan pendekatan stateless. Data keranjang selalu dikirim secara utuh oleh browser pada setiap request (melalui payload). Controller cukup mengambil data tersebut dari payload request ($request->cart) tanpa perlu membaca riwayat keranjang di session server.
3. Apa yang terjadi jika request dikirim ulang dengan payload yang sama?
   Respons dari controller akan selalu menghasilkan nilai dan perhitungan yang sama. Karena sistem bersifat stateless, server tidak menyimpan memori tentang request sebelumnya dan hanya menghitung ulang berdasarkan apa yang ada di payload yang diterima saat itu.
4. Mengapa keranjang kosong setelah halaman di-refresh?
   Karena variabel penyimpan keranjang (let cart = {};) hanya berjalan di memori sementara JavaScript. Variabel tersebut belum diinstruksikan untuk disimpan ke memori permanen di klien seperti localStorage ataupun cookie, sehingga akan terhapus dan kembali kosong (inisialisasi ulang) saat browser memuat ulang halaman
5. Apakah penggunaan middleware session Laravel berarti keranjang ini disimpan dalam session? Jelaskan.
   Tidak. Meskipun middleware session tetap aktif karena rute berada di web.php (digunakan untuk memvalidasi token CSRF), secara logika kode, CartController sama sekali tidak memanggil fungsi session() untuk menyimpan maupun membaca data keranjang tersebut
6. Apa risiko keamanan jika server mempercayai harga yang dikirim browser?

Risiko keamanannya adalah manipulasi data (tampering). Pengguna dapat dengan mudah memodifikasi payload JSON melalui DevTools atau Postman sebelum dikirim ke server. Jika server mempercayai harga dari browser, pembeli bisa mengubah harga produk menjadi Rp0. Oleh karena itu, controller hanya menerima ID dan kuantitas (jumlah) dari browser, lalu melakukan perkalian menggunakan harga asli yang berada di database atau array katalog di backend.

LESSON LEARNED

Saya belajar artsitektur stateless, ternyata server bertindak pasif dan tidak memiliki Riwayat data (session), oleh karena itu beban pengelolaan data bergeser ke sisi pengguna yang diharuskan untuk mengirim seluruh status data keranjang pada setiap request HTTP POST.

LINK GITHUB : https://github.com/Rafreaakk/Proyek-3-Week-5.git